Devoir de programmation :

Génération aléatoire d'arbres binaires

David VADIMON (21207099)
Abdullah AL MAMUN (21124650)
Yahya MUDALLAL (21238547)

UE d'Ouverture 2025/2026

Table des matières

Table des matières 2

Partie 1 : Implémentations des algorithmes 3

Partie 1.1 : Algorithme de Rémy 3

Question 1.1 3

Question 1.2 3

Question 1.3 3

Question 1.5 5

Question 1.6 5

Partie 1.2 : Arbres de type ABR 6

Question 1.7 6

Partie 2 : Expérimentations 7

Formes des arbres obtenus par les 2 algos 8

Comparaison des temps d'exécution des algorithmes 9

Comparaison des hauteurs d'arbres 10

Comparaison des largeurs d'arbres 11

Comparaison des tailles des sous-arbres gauches 12

Comparaison de la profondeur moyenne des feuilles : 14

Partie 1 : Implémentations des algorithmes

Partie 1.1 : Algorithme de Rémy

Question 1.1

> Voir fichier '*Arbre.ml*' pour l'implémentation.

Nous avons choisi d'avoir une structure à la fois utilisée pour l'algo de Rémy, ainsi que pour l'algo sur les arbres de type ABR, avec **2 *type*** :

- **noeud :** représente un nœud dans notre arbre binaire, identifié de façon unique par un id (commençant à 1). Peut être parmi **2 *record*** :
  - **Empty :** une feuille, ayant en champs :
    id (int) : l'identifiant de la feuille.
    p (noeud) : une référence sur le nœud parent, si il y en a un.
  - **Noeud :** sous-entendu interne, ayant en champs :
    id (int) : l'identifiant du nœud interne.
    g (noeud) : une référence sur l'enfant gauche (il y en a forcément un).
    d (noeud) : une référence sur l'enfant droit (il y en a forcément un).
    p (noeud) : une référence sur le nœud parent, si il y en a un.

- **arbre :** représente la structure de notre arbre binaire, possède en champs
  racine (noeud) : une référence sur le nœud racine de l'arborescence.
  noeuds (noeud array) : le tableau de tous les nœuds (internes + feuilles) de l'arbre.
  feuilles (noeud array) : le tableau de toutes les feuilles de l'arbre.

À noter que nos champs référents d'autres nœuds sont *mutable*, de sorte à ne pas avoir à reconstruire une partie de l'arborescence à chaque modification, et donc afin de respecter la contrainte d'avoir les algos en temps linéaire.

Question 1.2

> Voir fichier '*Remy.ml*' pour l'implémentation.

Question 1.3

Montrons que notre algo de Rémy construit en temps linéaire un arbre binaire de taille n (en nombre de nœuds internes), quelque soit la mesure de complexité :

L'algo commence par appeler la fonction '***creeArbreRemy***', celle-ci :

- **est appelée au total n fois**, le premier appel étant fait dans '*algoRemy*', les autres étant récursifs dans '*creeArbreRemy*'. À chaque appel on effectue **1 comparaison** pour savoir si l'on est dans le cas de base ou le cas général.

- dans le cas de base, **si n = 0** :
  Notre structure d'arbre binaire réduit à une feuille en racine est initialisée :
  En **nombre d'écritures dans un tableau** : **O(n)** car on initialise 2*n+1 cases pour le tableau des nœuds, et n+1 cases pour le tableau des feuilles, en y écrivant des nœuds temporaires, et on effectue 2 écritures en plus : 1 pour mettre la feuille en case 0 du tableau des noeuds + 1 pour mettre la feuille en case 0 du tableau des feuilles.
  Pour toutes les **autres mesures** : s'effectue en **O(1)**.

- dans le cas général, **si n > 0** :
  Après avoir récupéré l'arbre créé récursivement de taille n-1.
  On choisit de manière aléatoire un nœud de l'arbre, en **O(1).**
  On s'intéresse alors à la complexité d'un appel de 'ajouteNoeudRemy' pour modifier le noeud ciblé : cet appel
  - initialise 1 nouveau nœud interne (1 affectation pour la variable + 4 affectations de champs) et 1 nouvelle feuille (1 affectation pour la variable + 2 affectations de champs) = 8 = **O(1) affectations.**
  - 1 comparaison pour faire le pile/face = **O(1) comparaison**.
  - modifie en place l'arbre, via les références des noeuds manipulés (1 affectation dans '*remplacerEnfant*' + 2 * 1 affectation dans '*mettre_parent*') = 3 = **O(1) affectations.** En plus de 2 = **O(1) comparaisons** (sans compter le match) dans '*remplacerEnfant*'.
  - ajoute les 2 nouveaux nœuds dans les tableaux de la structure (2 écritures dans le tableau des nœuds, et 1 écriture dans le tableau des feuilles) = 3 = **O(1) écritures dans un tableau.**
  - par ailleurs, en considérant 1 comparaison par cas dans un *match*, on a en plus effectué dans le pire cas 7 = **O(1) comparaisons**, en comptant aussi celles dans les fonctions appellées.

  Complexité de l'appel de 'ajouteNoeudRemy' donc en **O(1).**
  On récupère les résultats de l'appel dans 3 variables = **O(1) affectations**.
  On termine par éventuellement mettre à jour la racine = 1 affectation, en **O(1)**, avant de renvoyer l'arbre construit avec n nœuds internes.

Donc notre algo, qui appelle la fonction ‘creeArbreRemy’, a une complexité :
En nombre d’écritures dans un tableau : O(n) cas de base + (n-1) * O(1) dans le cas général = **O(n).**
En nombre de comparaisons : n * (1 + O(1)) = **O(n).**
Pour toutes les autres mesures : O(1) cas de base + (n-1) * O(1) dans le cas général = **O(n).**

(On a pas compté les 3 dernières affectations, suite aux résultats de l’appel à ‘creeArbreRemy’ dans ‘algoRemy’, en O(1) donc non significatif)

Ainsi, notre algo est en O(n) quelle que soit la mesure de complexité.

Question 1.5

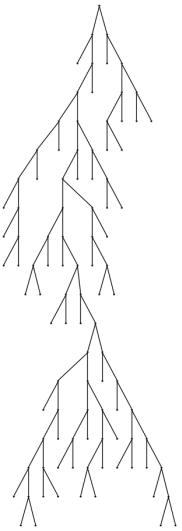
> Appeler le fichier ‘*executables/Remy.ml*’ avec n en paramètre pour tester.

Pour les .dot générés, par exemple :

n = 5

n = 20

n = 50



Question 1.6

> Voir fichier ‘*EcritureDot.ml*’ pour l'implémentation de l’encodage en .dot. À chaque exécution de nos algos l’arbre résultant est sauvegardé dans un .dot en appelant une fonction dans ce fichier.

Partie 1.2 : Arbres de type ABR

Question 1.7

> Voir fichier '*ABR.ml*' pour l'implémentation.

Montrons que notre algo avec arbres de type ABR construit en temps linéaire un arbre binaire de taille n (en nombre de nœuds internes), quelque soit la mesure de complexité :

Le code étant quasiment identique à celui utilisé pour Rémy, le preuve l'est aussi.

L'algo commence par appeler la fonction '***creeArbreABR***', celle-ci :

- **est appelée au total n fois**, le premier appel étant fait dans '*algoABR*', les autres étant récursifs dans '*creeArbreABR*'. À chaque appel on effectue **1 comparaison** pour savoir si l'on est dans le cas de base ou le cas général.

- dans le cas de base, **si n = 0** :
  Justification identique à l'algo de Rémy.
  En **nombre d'écritures dans un tableau** : **O(n)**.
  Pour toutes les **autres mesures** : **O(1)**.

- dans le cas général, **si n > 0** :
  Après avoir récupéré l'arbre créé récursivement de taille n-1.
  On choisit de manière aléatoire une feuille de l'arbre, en **O(1).**
  On s'intéresse alors à la complexité d'un appel de 'ajouteNoeudABR' pour modifier la feuille ciblée : cet appel effectue quasiment le même traitement que pour l'algo de Rémy, en sachant qu'**ici il n'y a pas la comparaison du pile/face**, mais sinon **le reste de la justification est identique** en terme de complexité dans le pire cas.
  Complexité de l'appel de 'ajouteNoeudABR' donc en **O(1).**
  On récupère les résultats de l'appel dans 3 variables = **O(1) affectations**.
  On termine par éventuellement mettre à jour la racine, avec **1 comparaison** et **1 affectation**, en **O(1)**, avant de renvoyer l'arbre construit avec n nœuds internes.

Donc notre algo, qui appelle la fonction 'creeArbreABR', a une complexité :
En nombre d'écritures dans un tableau : O(n) cas de base + (n-1) * O(1) dans le cas général = **O(n).**
En nombre de comparaisons : n * (1 + O(1)) = **O(n).**
Pour toutes les autres mesures : O(1) cas de base + (n-1) * O(1) dans le cas général = **O(n).**

(On a pas compté les 3 dernières affectations, suite aux résultats de l'appel à 'creeArbreABR' dans 'algoABR', en O(1) donc non significatif)

Ainsi, notre algo est en O(n) quelle que soit la mesure de complexité.

Partie 2 : Expérimentations

Dans cette partie, nous comparons divers paramètres des arbres générés par les algorithmes **ABR** et **Rémy** en fonction de la taille des arbres (en nombre de nœuds internes). Pour chaque taille d'arbre considérée, les résultats expérimentaux correspondent à la moyenne des données sur 100 arbres, ceci afin de minimiser les facteurs aléatoires.

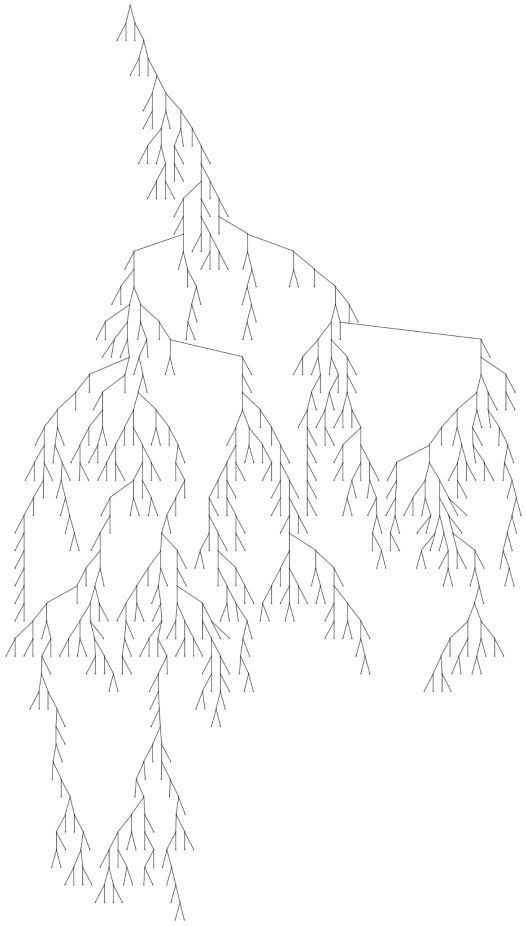
Les expérimentations ont été effectuées sur des tailles allant de 1 000 à 100 000 nœuds, par pas de 1 000. Toutefois, pour plus de clarté, le tableau ci-dessous ne présente qu'un extrait des résultats avec un échantillonnage tous les 20 000 nœuds internes.

taille (nb de noeuds internes)	temps **ABR** (s)	temps **Rémy** (s)	largeur **ABR**	largeur **Rémy**	hauteur **ABR**	hauteur **Rémy**	taille sous arbre gauche **ABR**	taille sous arbre gauche **Rémy**	profondeur moyenne des feuilles **ABR**	profondeur moyenne des feuilles **Rémy**
20 000	0.008396	0.011947	3 798	246	34	485	19 049	17 042	19.060961	244.458929
40 000	0.021433	0.026592	7 320	339	37	716	36 972	37 719	20.343292	363.075298
60 000	0.036424	0.043170	10 822	424	38	849	62 966	58 085	21.163176	428.998729
80 000	0.064234	0.074407	14 142	472	39	1 031	81 465	83 237	21.689991	530.350449
100 000	0.080412	0.093433	17 678	541	40	1 125	102 988	111 983	22.086421	563.950809

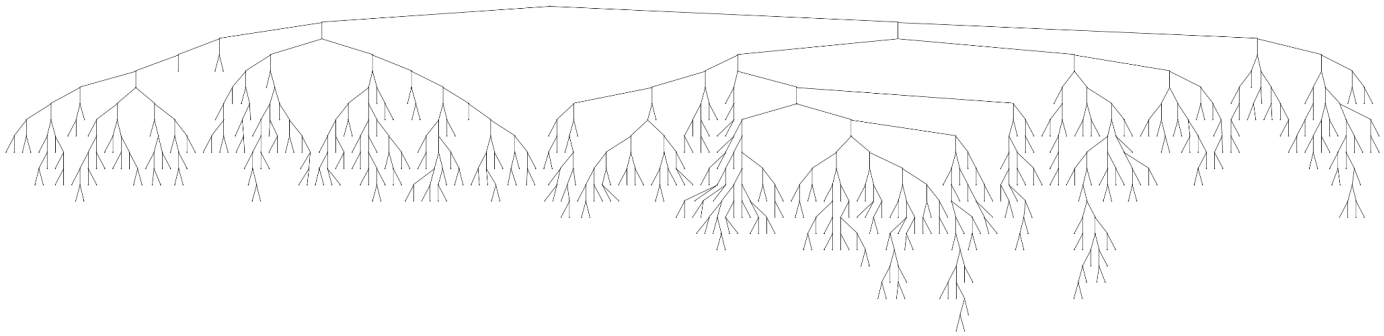
Formes des arbres obtenus par les 2 algos

Pour avoir une idée de la forme générale des arbres générés par les 2 algorithmes, voici 2 arbres générés avec une taille de 400 (en nombre de nœuds internes). On remarque que conformément aux algos, Rémy va avoir tendance à créer un arbre étiré en hauteur, tandis que ABR va étirer en largeur.

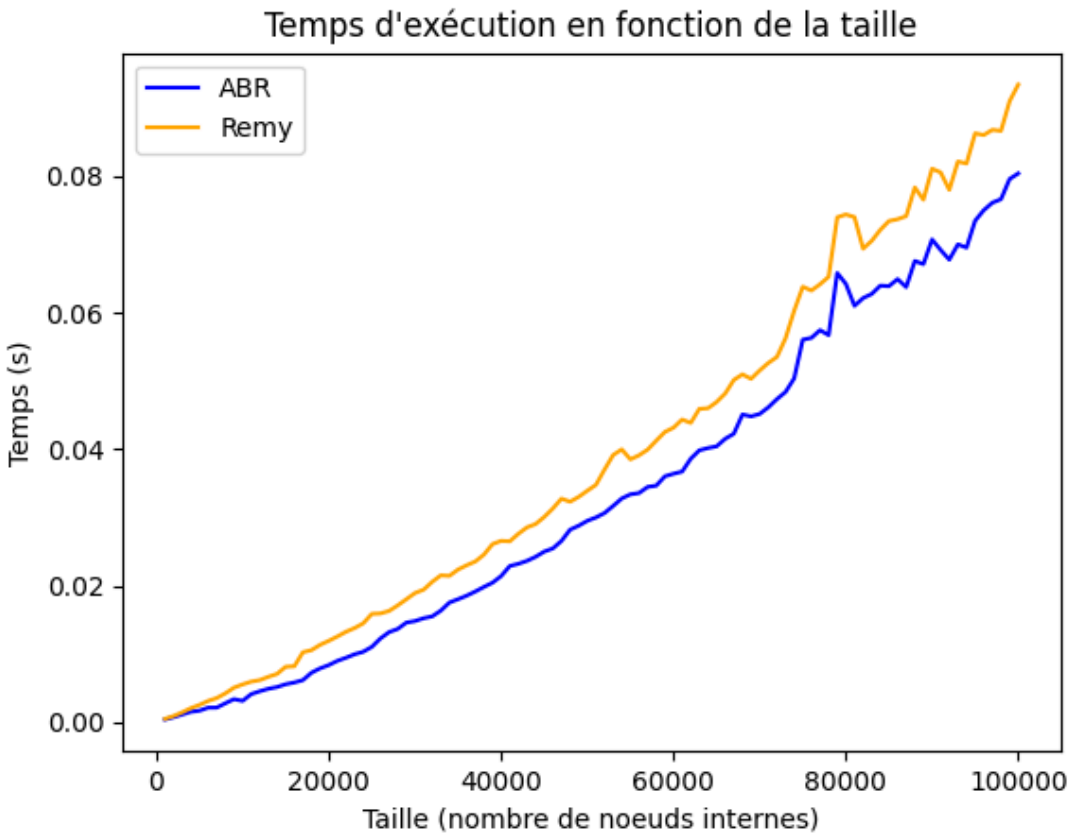
Rémy



ABR



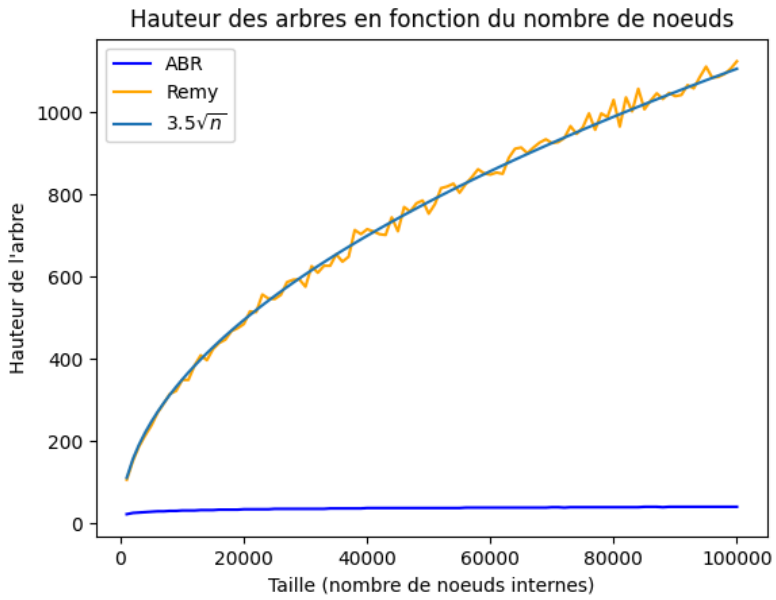
Comparaison des temps d'exécution des algorithmes

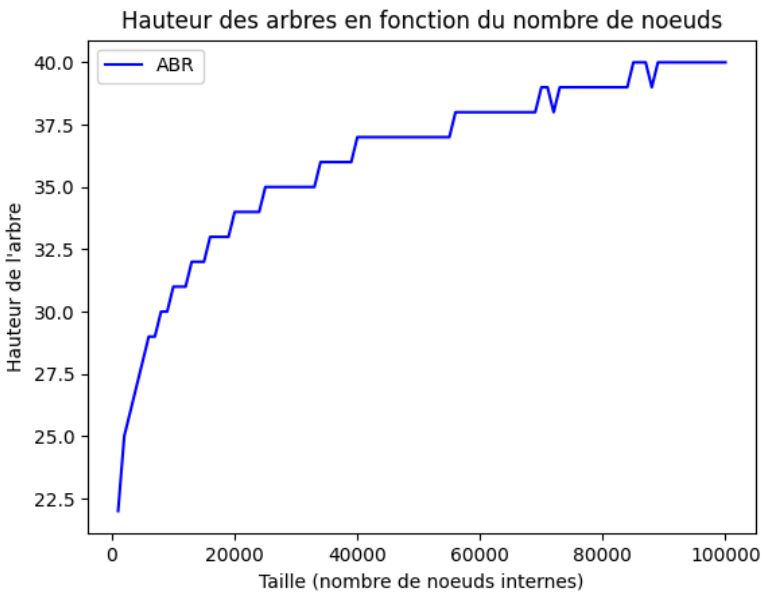


On constate qu'en accord avec les études théoriques de leurs complexités, les algorithmes de **Rémy** et de l'arbre de type **ABR** semblent effectivement suivre une **croissance assez linéaire** en fonction du nombre total de nœuds internes jusqu'à une taille d'environ 80 000 nœuds internes. À partir de ce nombre, la croissance semble néanmoins augmenter de manière un peu plus élevée que précédemment, ce qui est difficilement explicable compte-tenu du fait que nos algos soient théoriquement linéaires en le nombre de nœuds internes, et qui peut donc potentiellement s'expliquer par l'exécution d'autres choses par le système à ce moment, suggéré par le pic à 80 000.

Nous remarquons également que, bien que les courbes suivent une croissance similaire, l'algorithme des **ABR est légèrement plus performant que celui de Rémy**. Cette différence est due au fait que l'algorithme de Rémy, à chaque insertion de nœud, nécessite un tirage aléatoire (pile ou face) et une comparaison de plus que l'insertion dans un ABR.

Comparaison des hauteurs d'arbres





Sur le graphique de gauche, nous observons une **très grande différence de hauteur** entre les arbres créés par l'algorithme de **Rémy** et ceux créés par l'algorithme **ABR**, comme illustré par les cas des images des arbres en introduction des expérimentations.

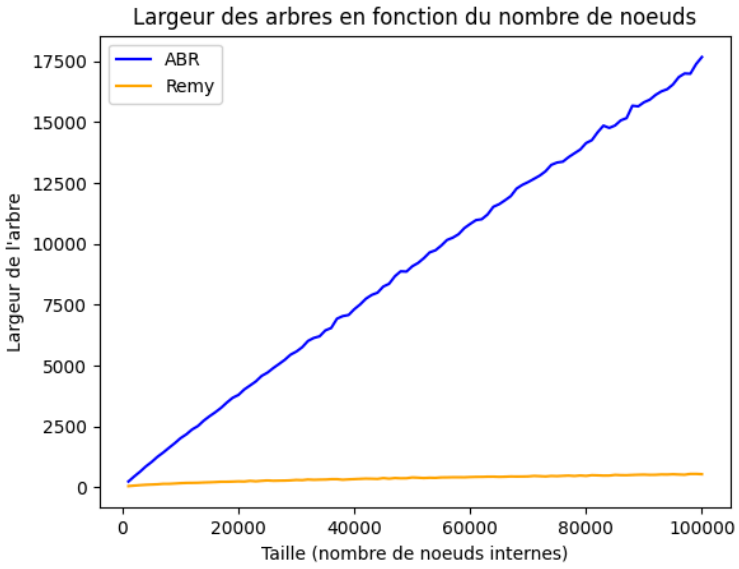
En effet, les arbres de **Rémy** ont une **hauteur bien supérieure**. Cela s'explique notamment ainsi : si l'on voit notre arbre de Rémy comme un ensemble de chemins, partants de la racine et allant jusqu'aux feuilles, alors il est évident que plus un de ces chemins est long, plus il a de noeuds, et donc plus il y a de chances que l'algo insère un nouveau noeud (interne) dans ce chemin, en choisissant un noeud en faisant partie. Si ce chemin s'avère être le plus long, et dont la longueur définie la hauteur de l'arbre, alors le choix d'un nœud de ce chemin, impliquant l'insertion d'un nouveau nœud interne dedans, contribue à augmenter la hauteur de l'arbre. On remarque donc que **l'algorithme favorise le déséquilibre de l'arbre** en favorisant l'expansion des plus longs chemins, comme justifié auparavant,

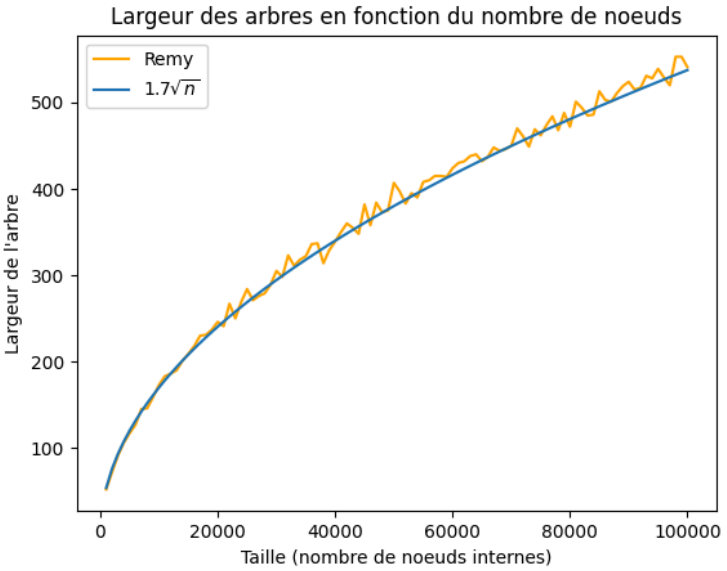
D'autre part, toujours sur le graphique de gauche, l'évolution de la hauteur des arbres de **Rémy** suit très clairement une racine carrée de la taille de l'arbre en nombre de nœuds internes, comme on peut le voir avec la superposition de la fonction $3.5\sqrt{n}$, donnant donc une **hauteur en $O(\sqrt{n})$**.

En revanche, la **hauteur des arbres ABR** s'apparente davantage à une **courbe logarithmique**, symbolisée sur le graphique de droite par une croissance rapide au début qui tend à se stabiliser par la suite, et qui est cohérente avec le fait que ces arbres sont très équilibrés, dont l'analyse de la largeur par la suite permet d'expliquer la raison de cet équilibre.

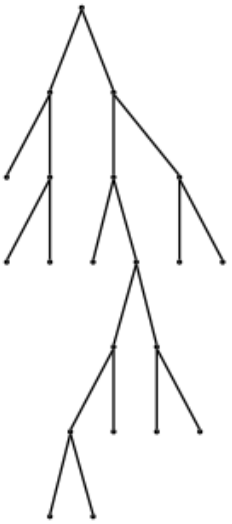
Comparaison des largeurs d'arbres

Dans ces graphiques, on définit la **largeur** d'un arbre par **le nombre maximal de nœuds à une même profondeur**.





Par rapport à précédemment, on peut voir ici en quelque sorte l'effet inverse sur le graphique de gauche : les arbres **ABR ont une largeur bien supérieure à celle des arbres de Rémy**.

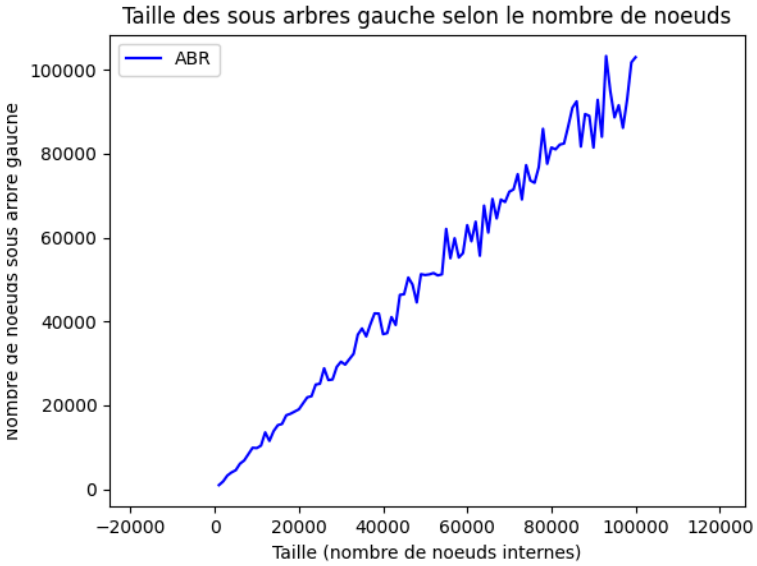


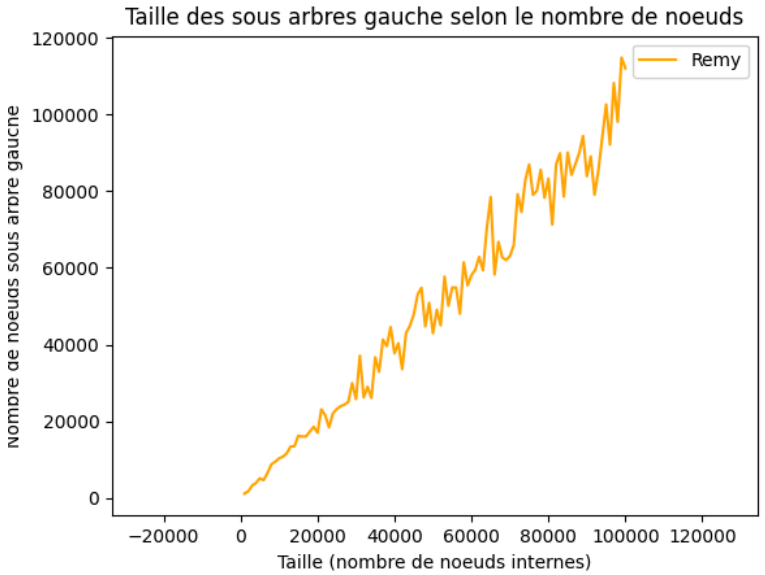
La largeur des ABR évolue très clairement de façon **linéaire** selon le nombre de nœuds internes, et s'explique notamment ainsi : plus un ABR va créer un arbre déséquilibré, par choix successifs de feuilles à une profondeur élevée, et plus il y aura de feuilles présentes dans des profondeurs inférieures, et donc susceptibles d'être choisies pour les futures insertions.
Sur l'exemple de droite, pour un ABR de taille 10, on voit donc qu'à partir d'un certain déséquilibre il y aura plus de chances que l'algo choisisse une feuille à une profondeur inférieure à la profondeur maximum, et en choisissant une de ces feuilles il introduira alors une feuille supplémentaire à une profondeur inférieure à la profondeur maximum.
Ainsi, l'algo ABR va de plus en plus compenser le déséquilibre en ayant de plus en plus de chances d'insérer par la suite dans les profondeurs moins élevées, en y ayant de plus en plus de feuilles après insertions.
On remarque donc qu'ici **l'algorithme favorise l'équilibre de l'arbre**.

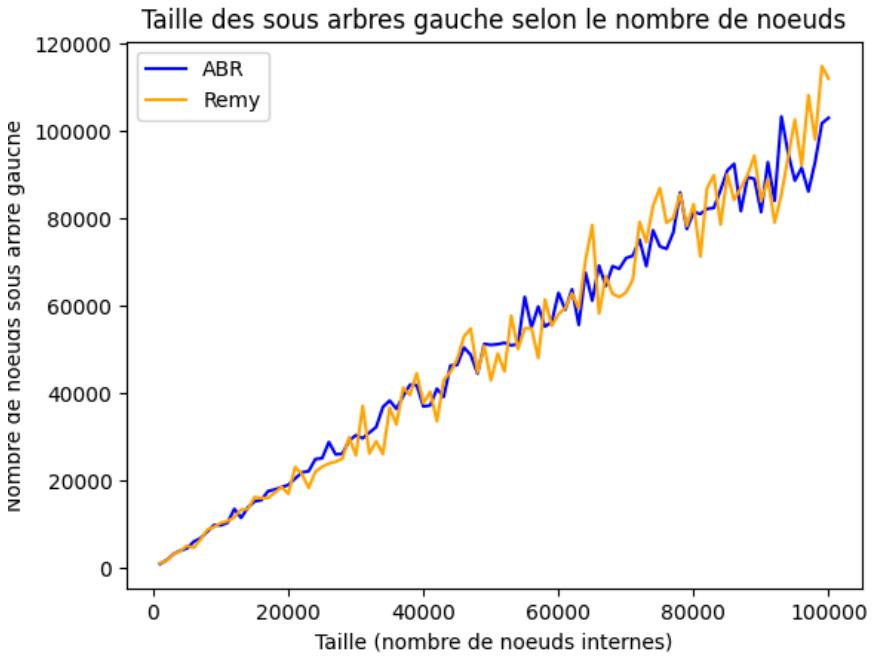
À l'inverse sur le graphique de droite, on peut supposer visuellement, et en cohérence avec l'analyse des hauteurs, que la **largeur des arbres de Rémy** évolue selon la racine carrée du nombre de nœuds internes. On remarque que c'est effectivement le cas avec la superposition de la courbe de la largeur avec la fonction 1.7√n, donnant donc une **largeur en O(√n)**.

Comparaison des tailles des sous-arbres gauches

Nous ne regardons ici que la taille du **sous-arbre gauche de la racine**, exprimée en nombre **total** de nœuds (internes + feuilles), sur l'axe des ordonnées.







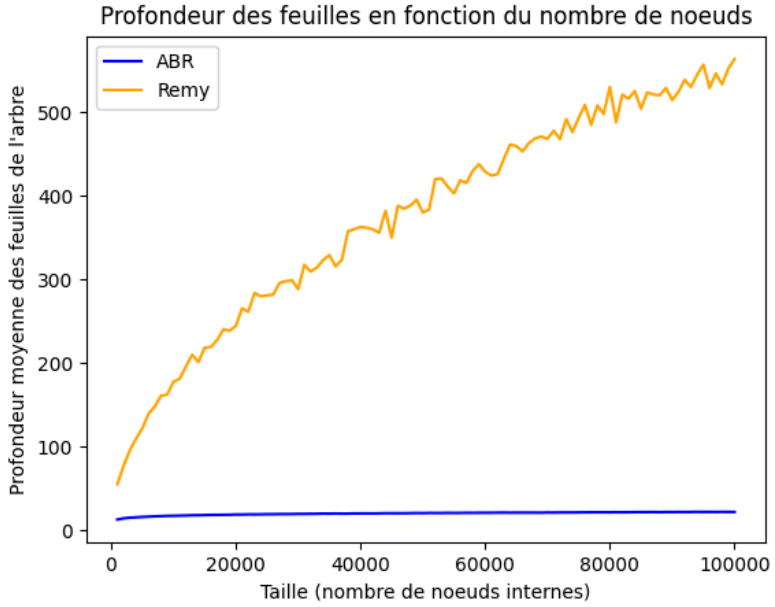
Nous pouvons constater sur ce dernier graphique que les **tailles des sous-arbres gauches de la racine des arbres ABR et de Rémy sont similaires**.
Leurs évolutions sont **linéaires** en fonction du nombre de nœuds internes, et l'on trouve presque une relation de correspondance 1:1 entre les axes x:y, expliqué par le fait que pour un arbre de taille n, en nombre de noeuds internes, il possède alors 2*n+1 noeuds au total, et devrait donc dans un cas équilibré avoir (2*n+1)/2 nœuds dans son sous-arbre gauche de la racine. Or (2*n+1)/2 ~= n, donc on a environ une correspondance n:n.

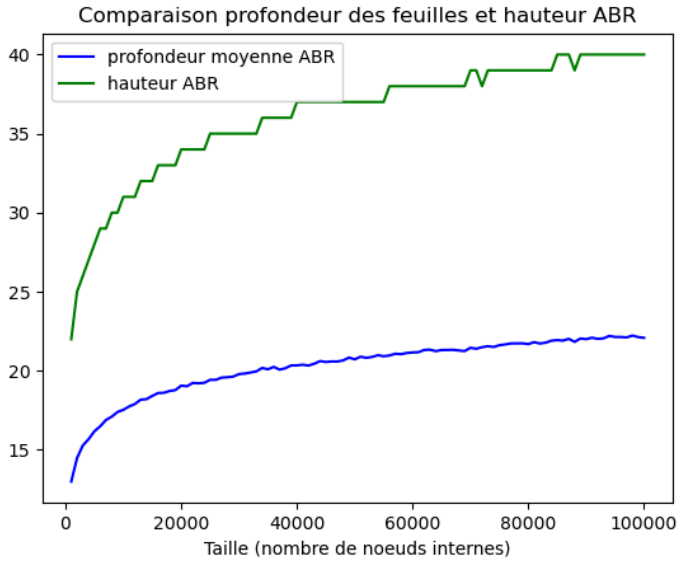
Concernant **l'évolution des courbes**, on remarque que les 2 courbes ont tendance à avoir quelques **fluctuations**, mais que celle de Rémy fluctue plus que celle des arbres ABR, cet effet semblant aussi augmenter pour Rémy avec l'augmentation de la taille en x.

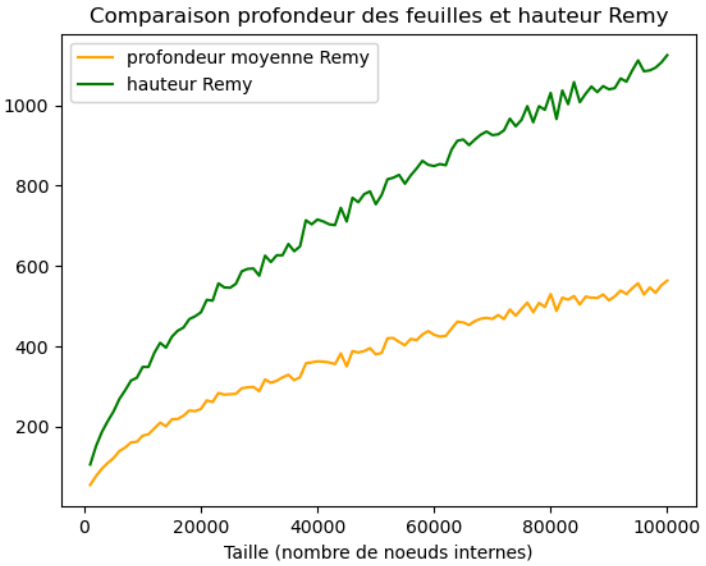
En parlant dans un premier temps de la **fluctuation de l'algo ABR**, sur le graphe de gauche, même si celui-ci favorise des arbres équilibrés, ceux-ci ne le sont jamais parfaitement et donc nous font observer quelques légères fluctuations. On peut néanmoins émettre une hypothèse : à défaut d'être parfaitement équilibrés, ces arbres tendent vers l'équilibre, et donc en faisant une moyenne non pas sur 100 arbres de chaque taille, mais sur bien plus, alors la courbe obtenue devrait être soumise à moins de fluctuations.

Pour les **fluctuations de l'algorithme de Rémy**, sur le graphe de droite, il a tendance à créer des arbres plus excentrés que l'algorithme ABR, en la racine, ce qui est normal compte-tenu de notre analyse précédente montrant que l'arbre de Rémy va avoir tendance à être déséquilibré. Le nombre de nœuds augmentant en x, le déséquilibre aussi, et donc les fluctuations deviennent plus fortes.

Comparaison de la profondeur moyenne des feuilles :







Le 1er graphique nous montre que les **feuilles des arbres ABR sont situées à une profondeur moyenne beaucoup plus faible que celles des arbres de Rémy**. Cela reste cohérent avec les résultats obtenus précédemment sur la hauteur et la largeur : les arbres ABR ont tendance à être plus larges et moins hauts que les arbres de Rémy.

Sur le graphe de gauche, la courbe de la **profondeur moyenne des arbres ABR** évolue de manière **logarithmique** en fonction de la taille, et de manière plus prononcée que la hauteur.

Tandis que sur le graphe de droite, **celle des arbres de Rémy** évolue selon la racine carrée de la taille, et **est en O(√n)** car bornée par sa hauteur, elle-même en O(√n).

Cela rend donc les arbres **ABR en moyenne 20 fois plus efficaces que les arbres de Rémy**, pour des **opérations nécessitant de parcourir l'arbre**, telle qu'une recherche par exemple.